Solutions

Pour les interactions avec les forges, nous allons ici toujours utiliser Gitlab. Pour GitHub, les commandes sont identiques en remplaçant `gitlab.com` par `github.com`. Pour cet exercice, notre dépôt git de départ ressemble à cela :

Les 4 caractères à l'intérieur de chaque nœud sont le début du hash de chaque commit. 4 est le nombre de caractères minimum à partir duquel il est possible d'identifier un commit git, c'est donc un raccourci fréquemment utilisé pour y référer.

Exercice 1

1. Cloner le dépôt `Rumengol/formation_M4DI_2024`

```
git clone git@gitlab.com/Rumengol/formation_M4DI_2024
```

Cette commande crée un dossier `formation_M4DI_2024` sur la branche `main` connectée au dépôt distant, et télécharge tous les fichier correspondants au dernier commit sur cette branche. Bien sûr, l'historique git est conservé.

> Vous pouvez explorer un peu les commits et le contenu des différentes branches. Remarquez-vous quelque chose de particulier concernant la rigueur des deux branches feature ? L'une d'entre elle respecte plus les bonnes pratiques que l'autre.

2. Rebase la branche `feat_revcomp` sur main

Pour le moment, il n'y a qu'une seule branche locale :

```
(main)$ git branch
* main
```

En revanche, les branches distantes sont déjà présentes :

```
(main)$ git branch -r # permet de voir les branches distantes
origin/HEAD -> origin/main
origin/main
origin/feat_revcomp
origin/feature_1
```

Attention, ces branches sont principalement *read only*. Elles sont utilisées dans la plomberie interne de git pour communiquer avec le dépôt distant (ici, `origin` est le nom de la remote connectée au dépôt GitLab). A moins de savoir ce que l'on fait, mieux vaut ne jamais interagir avec elles directement.

Elles nous indiquent cependant que nous possédons deux branches dont les données ont été téléchargées, mais pas appliquées localement. Pour cela, il suffit de se déplacer sur ces branches, sans la remote `origin/` :

```
(main)$ git checkout feat_revcomp
branch 'feat_revcomp' set up to track 'origin/feat_revcomp'.
Switched to a new branch 'feat_revcomp'

(feat_revcomp)$ git log --oneline --graph
* a6cdc24 (HEAD -> feat_revcomp, origin/feat_revcomp) Created revcomp.py
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

Notez qu'ici, on ne voit pas la branche `main`. En effet, celle-ci et la branche `rev_comp` sur laquelle nous somme actuellement ont divergé au commit précédent, `862ae07`. La commande `git log` ne montre que l'historique de la branche actuelle.

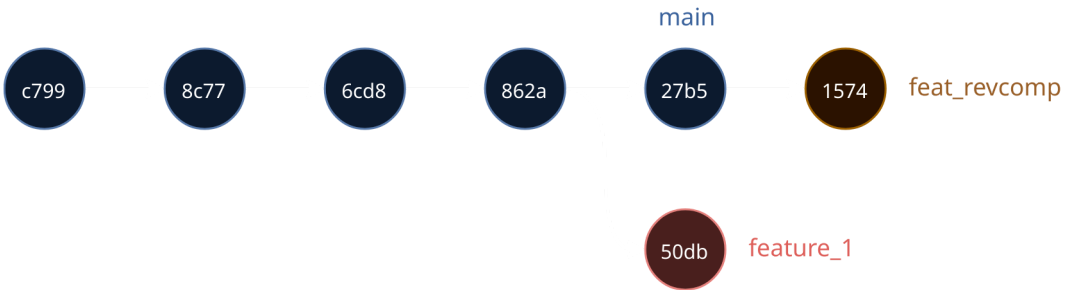
La commande `git rebase` permet de déplacer une branche au bout d'une autre et d'obtenir un historique linéaire.

```
(feat_revcomp)$ git rebase main
Successfully rebased and updated refs/heads/feat_revcomp.

(feat_revcomp)$ git log --oneline --graph
* 157497d (HEAD -> feat_revcomp) Created revcomp.py
* 27b5bbd (origin/main, origin/HEAD, main) Filled README
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

Nous avons déplacé le contenu de `feat_revcomp` au bout de la branche `main`. Désormais, nous voyons la branche `main` dans l'historique, car son extrémité (le

commit vers lequel `main` pointe) est intégré à `feat_revcomp`. Notre arbre ressemble à ceci :



Notez que le hash du dernier commit de feat_revcomp a changé : à cause du `rebase`, il ne possède plus le même parent qu'auparavant. Puisque son contenu a changé, son hash aussi. Si vous utilisez les hash des commits plutôt que des références relatives comme `HEAD~2` par exemple, il faut faire attention à ce type de détail.

Du point de vue de `main`, rien n'a changé :

```
(feat_revcomp)$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

(main)$ git log --oneline --graph
* 27b5bbd (HEAD -> main, origin/main, origin/HEAD) Filled README
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

Pour mettre `main` à jour, il faut effectuer un merge **fast-forward**. Contrairement à un merge classique, qui crée un commit contenant le résultat du merge, le fast-forward permet simplement à la tête d'une branche d'avancer jusqu'au commit spécifié. Ce n'est possible que dans un historique linéaire, comme nous avons ici.

```
(main)$ git merge feat_revcomp
Updating 27b5bbd..157497d
Fast-forward
 revcomp.py | 7 +++++++
 1 file changed, 7 insertions(+)
 create mode 100644 revcomp.py

(main)$ git log --oneline --graph
* 157497d (HEAD -> main, feat_revcomp) Created revcomp.py
* 27b5bbd (origin/main, origin/HEAD) Filled README
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
```

```
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

Nous n'avons mis à jour que `main` et pas le dépôt distant, c'est pourquoi `origin/main` reste au même endroit. Si nous lancions la commande `git push`, la branche distante et le dépôt s'actualiseraient pour rejoindre le commit `157497d`.

A noter qu'il est tout de même possible de créer un merge commit lorsqu'un fast-forward est possible, avec l'option `--no-ff`. Cela peut être préférable en fonction de la philosophie en usage, car le fast-forward perd le contexte du merge dans l'historique.

3. Merge la branche `feature_1` avec main

De la même manière que pour `feat_revcomp`, pour obtenir et manipuler le contenu de la branche `feature_1` en local, il nous suffit de la checkout:

```
(main)$ git checkout feature_1
branch 'feature_1' set up to track 'origin/feature_1'.
Switched to a new branch 'feature_1'
```

Revenons sur `main` pour réaliser le merge.

```
(feature_1)$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

(main)$ git merge feature_1
```

Cette commande vous emmène dans l'éditeur du système (les plus courants sont **vi** et **nano**) pour écrire le message du commit de merge. Pour cette fois, vous pouvez accepter celui proposé par défaut et quitter l'éditeur en tapant `:wq` sur **vi** ou la commande `ctrl + s`, `ctrl + x` sur **nano**.

```
Merge made by the 'ort' strategy.
 file_v2.txt | 5 -----
 file_v3.txt | 5 +++++
 2 files changed, 5 insertions(+), 5 deletions(-)
```

```
 delete mode 100644 file_v2.txt
 create mode 100644 file_v3.txt

(main)$ git log --oneline --graph
*   6e9f520 (HEAD -> main) Merge branch 'feature_1'
|\
| * 50db10d (origin/feature_1, feature_1) changed logoden biniou into
lorem ipsum
* | 157497d (feat_revcomp) Created revcomp.py
* | 27b5bbd (origin/main, origin/HEAD) Filled README
|/
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

En effet, un nouveau commit a été généré automatiquement, contenant les informations du merge. Ce commit possède **2 parents**. La plupart du temps, cela n'a pas d'importance, sauf si vous utilisez l'opérateur relatif `HEAD^` (qui est équivalent à `HEAD^1`). Cet opérateur fait remonter au **premier parent** de HEAD, dans ce cas (pour le commit `6e9f`) au commit `1574`, tandis que `HEAD^2` fait remonter au **deuxième parent**, ici `50db`.

L'opérateur linéaire `HEAD~`, lui, remontera toujours l'historique de la branche actuelle sans prendre en compte les différents choix possible. Ici, `HEAD~` (ou `HEAD~1`) remonte à `1574`, `HEAD~2` à `27b5`, etc...

> N'hésitez pas à explorer les commits avec `git diff` pour voir ce qui change à chaque fois. Remarquez-vous une branche respectant plus les bonnes pratiques que l'autre ?

4. Revert le commit changeant le nom du fichier en `file_v3.txt`

Cette information n'est pas donnée dans l'historique. En effet, le commit `50db` contient simplement "changed logoden biniou into lorem ipsum". Pourtant, en observant les changements, ce commit change également le nom du fichier de `file_v2.txt` à `file_v3.txt`. C'est pour cela qu'il est important de faire des messages de commits exhaustifs, c'est à dire qu'ils mentionnent tous les changements effectués, sans nécessairement entrer dans les détails.

Dans tous les cas, nous avons désormais la référence du commit que nous voulons `revert`.

```
(main) git revert 50db
```

Comme avant, un éditeur s'ouvre pour que nous inscrivions le message de revert. De la même manière, nous pouvons garder le message par défaut et quitter l'éditeur.

```
2 files changed, 5 insertions(+), 5 deletions(-)
create mode 100644 file_v2.txt
delete mode 100644 file_v3.txt

(main)$ git log --oneline --graph
* cc360e0 (HEAD -> main) Revert "changed logoden biniou into lorem ipsum"
*   6e9f520 Merge branch 'feature_1'
|\
| * 50db10d (origin/feature_1, feature_1) changed logoden biniou into
lorem ipsum
* | 157497d (feat_revcomp) Created revcomp.py
* | 27b5bbd (origin/main, origin/HEAD) Filled README
|/
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

Notes qu'ici, les fichiers contenus dans les commits `1574` et `cc36` sont identiques, puisque ce dernier est l'exact opposé de `50db`, et que `6e9f` n'est que la fusion de deux branches. Par ailleurs, `git revert 6e9f -m 1` aurait eu le même effet, mais demande une option supplémentaire.

En effet, puisque `6e9f` est un merge commit, il possède deux parents. Seulement, git n'est pas capable de savoir quel est la branche principale entre les deux, et donc quelle partie du merge annuler. L'option `-m 1` indique de conserver le **premier parent** du merge commit, ici `1574`.

5. Reset l'historique deux commits en arrière

Finalement, nous considérons que le revert n'est pas satisfaisant pour l'historique, et qu'il est plus simple de reset complètement jusqu'à avant le merge. Comme nous ne

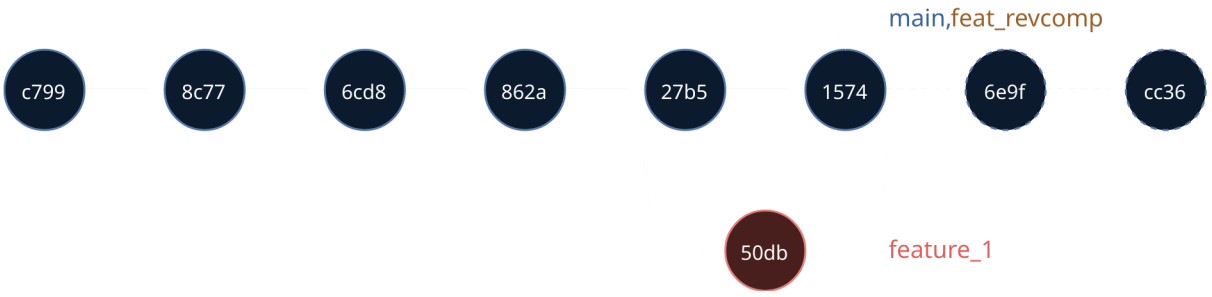
voulons conserver aucun des changements ajoutés, nous pouvons utiliser l'option `--hard`.

> Attention, assurez-vous toujours de ne pas avoir de fichiers non-trackés et non-commit lorsque vous faites un `git reset --hard`, autrement ils sont perdus pour toujours ! Si vous ne voulez pas créer de commit pour ces fichiers, `git add .; git stash` est votre ami.

Bien que nous connaissons le hash du commit vers lequel nous voulons retourner, nous allons cette fois utiliser les opérateurs de navigation relatives. En temps normal, utiliser un hash est toujours plus sûr, et plus précis. Il reste important de connaître les autres opérateurs :

```
(main)$ git reset --hard HEAD~2
HEAD is now at 157497d Created revcomp.py

(main)$ git log --oneline --graph
* 157497d (HEAD -> main, feat_revcomp) Created revcomp.py
* 27b5bbd (origin/main, origin/HEAD) Filled README
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```



6. Annulez le reset à l'aide de `git reflog`

Oups, c'était une erreur ! Heureusement, dans git rien n'est jamais perdu. Même dans le cas d'un `reset --hard`, si les données ont un jour été commit, elles sont quelque part dans la mémoire interne de git. Pour y accéder, il faut utiliser la commande `git reflog` :

```
(main)$ git reflog
157497d (HEAD -> main, feat_revcomp) HEAD@{0}: reset: moving to HEAD~2
cc360e0 HEAD@{1}: revert: Revert "changed logoden biniou into lorem ipsum"
6e9f520 HEAD@{2}: merge feature_1: Merge made by the 'ort' strategy.
157497d (HEAD -> main, feat_revcomp) HEAD@{3}: checkout: moving from
feature_1 to main
```

```
50db10d (origin/feature_1, feature_1) HEAD@{4}: checkout: moving from main
to feature_1
157497d (HEAD -> main, feat_revcomp) HEAD@{5}: merge feat_revcomp: Fast-
forward
27b5bbd (origin/main, origin/HEAD) HEAD@{6}: checkout: moving from
feat_revcomp to main
157497d (HEAD -> main, feat_revcomp) HEAD@{7}: rebase (finish): returning
to refs/heads/feat_revcomp
157497d (HEAD -> main, feat_revcomp) HEAD@{8}: rebase (pick): Created
revcomp.py
27b5bbd (origin/main, origin/HEAD) HEAD@{9}: rebase (start): checkout main
a6cdc24 (origin/feat_revcomp) HEAD@{10}: checkout: moving from main to
feat_revcomp
```

Nous retrouvons notre commit de revert, `cc360e0`, bien qu'il ait disparu de la vue classique `git log` ! Nous pouvons le restaurer à l'aide de `git reset` une nouvelle fois :

```
(main)$ git reset --hard HEAD@{1}
HEAD is now at cc360e0 Revert "changed logoden biniou into lorem ipsum"

(main)$ git log --oneline --graph
* cc360e0 (HEAD -> main) Revert "changed logoden biniou into lorem ipsum"
*   6e9f520 Merge branch 'feature_1'
|\
| * 50db10d (origin/feature_1, feature_1) changed logoden biniou into
lorem ipsum
* | 157497d (feat_revcomp) Created revcomp.py
* | 27b5bbd (origin/main, origin/HEAD) Filled README
|/
* 862ae07 Renamed file_v1 > file_v2 and added placeholder text
* 6cd81a1 Added file_v1.txt
* 8c77259 Create CONTRIBUTING.md
* c79994f Initial commit
```

Bien joué ! Avec ça, vous avez appris à utiliser les principales commandes de manipulation d'historique de git, ainsi que l'essentielle `git reflog` qui permet de réparer tout accident causé par une mauvaise commande Git.